Pre-Delinquency Intervention Engine for Real-Time Credit Risk Assessment Using Streaming ML and Ensemble Inference

Apurv D. Saktepar*, Nisha Y. Pragane*, Shantanu B. Kaute*, Soha I. Jamadar*

Dr. Sangita M. Jaybhaye*
*Department of Computer Science and Engineering (Artificial Intelligence),
Vishwakarma Institute of Technology, Pune, India
Email: {apurv.1252090011, nisha.1252090013, shantanu.1252090007,
soha.1252090024, sangita.jaybhaye}@vit.edu

***Abstract*—Traditionally, traditional methods for detecting delinquency in loans have relied on risk assessment of loans based on historical data. These have typically only been done at the start of a loan and then again periodically (i.e. during periodic reviews). The issue with this is that there is a significant time delay between the loan being issued and then reviewed at set intervals. As a result, any change in the customer's behavior leading up to default will have been ignored, as it occurred after the last periodic review. The authors describe PIE (Pre-Delinquency Intelligence Engine), a real-time risk intelligence platform built on machine learning that analyzes live transaction streams, generates a risk score for delinquent payments, and triggers workflows for intervention. The architecture of PIE includes a baseline scorer based on LightGBM that provides transactions with a score quickly without storing/transferring it between transactions, as well as a contextual model based on XGBoost, which incorporates evidence from the customer's historical behavior as well as from their current profile. The two models are balanced through a weighted fusion engine with an override for reviewers' override rules to ensure consistency in operations. Ongoing monitoring through PSI, KS, and Jensen-Shannon divergence will keep both models up-to-date; if either model reaches a threshold, it will automatically retrain itself. The authors tested their model with synthetic transactional data and with a cleaned version of real-world data. The authors found that PIE's overall scoring/initial contact with a customer takes less than 500 milliseconds and that the system can process more than 500 transactions per second per node. In addition, the ability of PIE to identify high-risk customers offers ease of intervention prior to default occurring. PIE shows how a composable streaming ML architecture can significantly change credit risk management from being reactionary (undoing problems after they have occurred) to being proactive (tools allowing financial institutions to make adjustments before issues arise).**

***Index Terms*—credit risk, delinquency prediction, gradient boosting, LightGBM, XGBoost, Redis streams, drift detection, ensemble learning, real-time ML, fintech**

I. Introduction

Credit markets worldwide manage billions of dollars worth of loan balances on an annual basis, with NPAs being a significant risk factor for banks and lending organizations [1]. Existing models to manage credit risk measure risk either when loans are first issued based on static bureau scores like FICO or CIBIL or periodically via batch processes every month or quarter [2]. Such an interval leaves a critical vulnerability since the behavior leading up to non-payment takes place continually within that period, leaving little room for action once detected [3].

Recent developments in infrastructure for stream processing, gradient boosted models, and MLOPs platforms have enabled the emergence of a novel approach called *continuous, event-driven credit risk scoring* which continuously tracks customers' activities and sends out notifications before a default event happens [4], [5]. Nevertheless, moving from such an approach to a production solution poses many difficulties including feature parity between real-time and batch flows, the problem of model reality alignment due to changing distributions of data, and routing the results of scoring to the right pipeline.

This paper describes PIE (Pre-Delinquency Intelligence Engine), a fully-fledged framework solving all these problems. The main contributions of PIE are the following:

1) A **two-step ensemble prediction workflow** leveraging LightGBM (for baseline score using transaction features) alongside XGBoost (for context model score using customer features), joined by a weighted disagreement mechanism.
2) A **Redis Streams-driven event consumption pipeline** supporting guaranteed ordered, exactly once, and isolated transactional processing with consumer groups.
3) A **multi-dimensional drift detection algorithm** integrating PSI, KS statistic test, and Jensen Shannon divergence with automatic retraining orchestration.
4) A **layered intervention orchestration solution** transforming risk scores into queues with human-in-the-loop decision making and logging.
5) A **holistic data analytics application** developed using React/TypeScript allowing risk ops teams to analyze model status, explore customer insights, and conduct interventions in real-time.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture thoroughly. Section IV explains the machine learning pipeline. Section V covers drift detection and retraining. Section VI describes intervention orchestration layer. Section VII presents experimental evaluation by the researchers. Section VIII discusses implications and limitations of the system. Section IX concludes the paper.

II. Related Work

A. Credit Scoring and Delinquency Prediction

Traditional credit scoring uses logistic regression and scorecard approaches on the data from bureaus [6]. Although these models are easily explainable and follow the regulations, they work only with static data and do not consider changes within a cycle. Machine learning techniques such as decision trees, random forest, gradient boosting, and neural networks have proven to consistently outperform logistic models in terms of delinquency predictions [3], [7]. From the ensembles, gradient boosted decision tree models (GBDT) became a dominant approach to work with tabular data from finance because of their flexibility to handle heterogeneity and calibration advantages [4], [5].

B. Streaming and Real-Time Machine Learning

Use cases for machine learning with streams include, but are not limited to, fraud detection [8], anomaly detection [9], and network intrusion detection [10]. Streaming technologies like Apache Kafka [11] and Redis Streams [12] have become key tools for building ordered and fast stream processing systems. Feature generation in real time, specifically, consistency issues of training and online features, is still a problem that needs more investigation, although some solutions come in form of feature stores [13].

C. Concept Drift and Model Monitoring

The performance of deployed machine learning models degrades when the data distribution changes, which is known as concept drift [14]. For example, in credit risk assessment, data distribution changes due to macroeconomic fluctuations, regulatory policy changes, fraud schemes evolving, and demographics changing [15]. Common statistical tests for detecting drift include the Kolmogorov-Smirnov (KS) test [16], Population Stability Index (PSI) [17], and Jensen-Shannon divergence (JS) [18].

D. Intervention and Collections Strategy

Modern risk-driven collections approaches have moved beyond basic dial queue techniques and into more advanced machine learning-driven approaches to prioritize customer interactions [2]. Contact propensities, timing of interventions, and optimization of multi-channel strategies are areas of current research interest [20]. Human-in-the-loop (HITL) processes for evaluating high-risk decision outcomes have been introduced due to explainability requirements and regulation [21].

The PIE framework differentiates itself through the combination of all these elements within a single, deployable platform.

III. System Architecture

PIE is structured as a layered, event-driven architecture. Fig. 1 presents the detailed ML pipeline architecture, and Fig. 2 presents the four-layer system view with parallel processing and intervention routing.

A. Layer 1: Real-Time Data Ingestion

The following are sources of the transaction events: (i) A configuration-based transaction simulator used for load testing and demos, (ii) An API-powered producer of events manually created or triggered by an upstream system, and (iii) Account event hooks from core banking integration. Events are then put into a Redis stream [12], which allows:

- **Persistent ordering**: All events are persisted in their append order, hence allowing the possibility of replaying.
- **Isolation via consumer groups**: Multiple workers will be able to pull events from the stream without duplicating.
- **Acknowledgments**: Events will not be removed from the pending-entry list until being acknowledged.

B. Layer 2: Parallel Processing Stage

After stream consumption, three paths are initiated in parallel within around 140 ms:

Path 1 – Storage (20ms): A FastAPI REST service stores the transaction in the PostgreSQL/PostgreSQL database table of customers' transactions via SQLAlchemy ORM and updates customer profiles in parallel.

Path 2 – Feature Engineering (40ms): A FastAPI feature service fetches the past 30 days of transactions of the customer from the Redis cache and performs feature engineering by deriving features like spend velocity, balance volatility, decline rate, DPD acceleration, and merchant entropy (further details provided in Section IV).

Path 3 – Risk Prediction (80ms): The inference service uses the features generated in Path 2 to predict risk using the base LightGBM model and the context-aware XGBoost model, storing the resulting risk score tuple in the PostgreSQL risk score table.

C. Layer 3: Intelligent Routing

The routing engine applies threshold-based if-else logic on the final fused score:

$$\text{Queue}(s) = \begin{cases} \text{LOG_EVENTS} & s < 30 \\ \text{LOW_RISK} & 30 \leq s < 60 \\ \text{HIGH_RISK} & 60 \leq s < 80 \\ \text{CRITICAL} & 80 \leq s < 90 \\ \text{VERY_CRITICAL} & s \geq 90 \end{cases} \tag{1}$$

Scores in the LOW_RISK range are logged passively in the database's logging table. Higher tiers trigger progressively more powerful intervention workflows (Section VI).

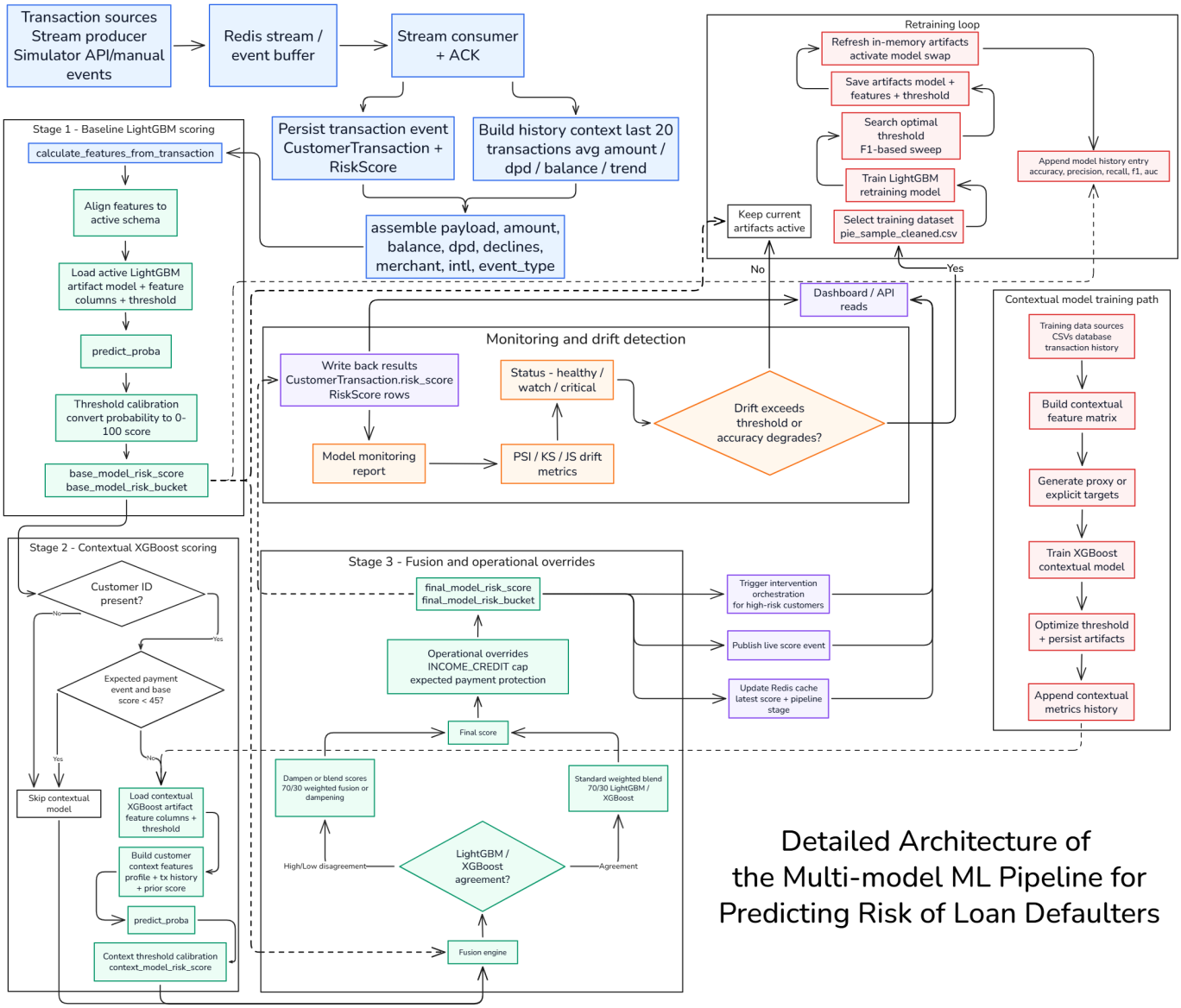

Fig. 1. Detailed multi-model ML pipeline architecture for PIE, showing Stage 1 (LightGBM baseline), Stage 2 (XGBoost contextual), Stage 3 (Fusion and Overrides), monitoring/drift detection loop, and contextual model retraining path.

D. Layer 4: Automated Agentic Interventions

There are four types of workflows (WF1-WF4), each corresponding to different risk categories. These workflows are: (WF1) passive monitoring where events are logged; (WF2) auto-generation of repayment schedules by using templates; (WF3) alerting and creating a case in a CRM application; and (WF4) ticket generation and subsequent manual intervention by contacting the customer. The outcome of whether the client pays off or replies will be one of the inputs in the feedback loop, which will help update the label library and begin re-training, along with improving the intervention approach, prioritization precision, and adaptability of the entire system to customer behavior.

IV. Machine Learning Pipeline

A. Feature Engineering

PIE computes two classes of features: transaction-level features for the baseline model and customer-level contextual features for the contextual model. Table I summarizes key features.

Key derived features are defined formally as follows. Spending velocity captures average transaction rate:

$$v_{\text{spend}} = \frac{\sum_{i=0}^{T} a_i}{T_{\text{active}}} \tag{2}$$

Balance volatility is normalized by the mean to enable

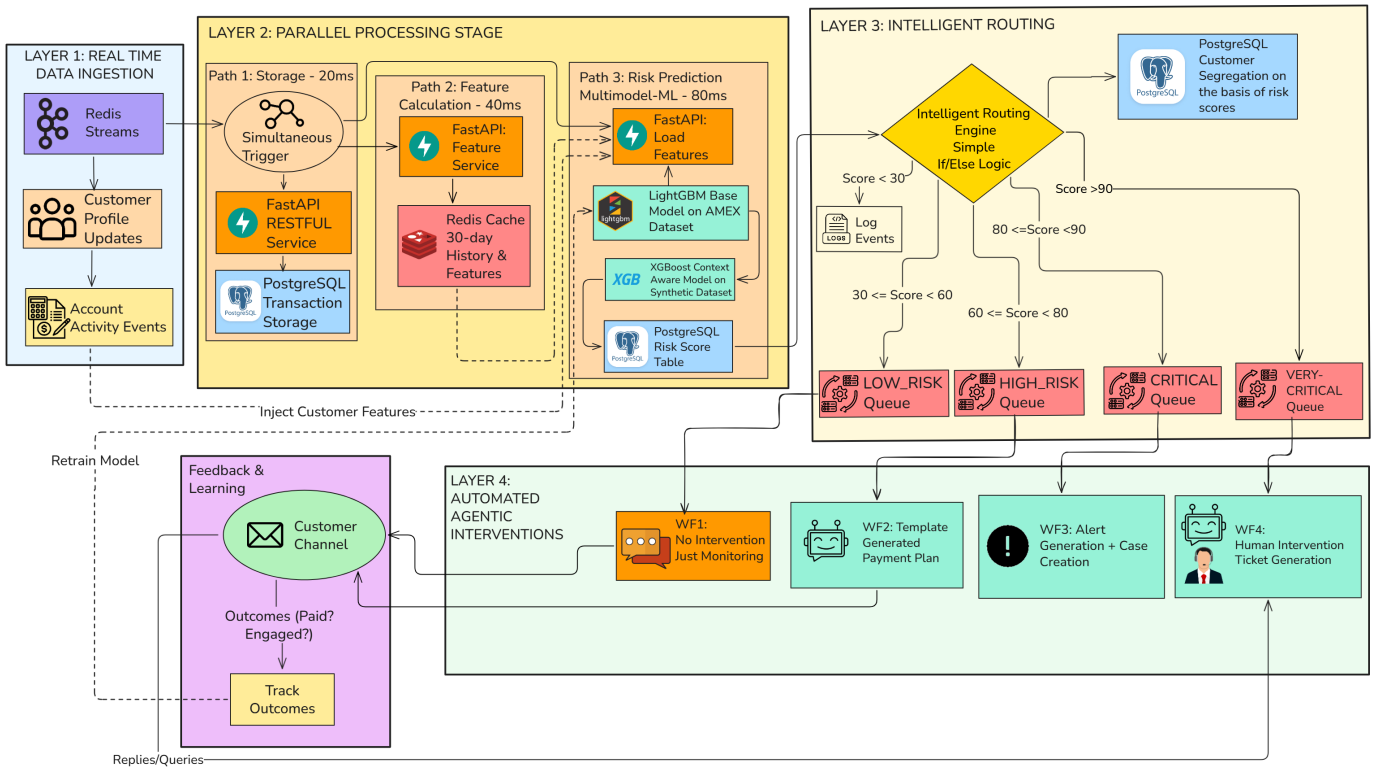

Fig. 2. Four-layer PIE system architecture: Layer 1 (Real-time Data Ingestion), Layer 2 (Parallel Processing — Storage, Feature Calculation, Risk Prediction), Layer 3 (Intelligent Routing by risk score tier), and Layer 4 (Automated Agentic Interventions with Feedback and Learning loop).

cross-customer comparison:

$$\sigma_{\text{bal}} = \frac{\text{std}(\mathbf{b})}{\text{mean}(\mathbf{b}) + \epsilon} \tag{3}$$

Merchant diversity is quantified using Shannon entropy over category proportions p_j:

$$H_{\text{merch}} = -\sum_j p_j \log p_j \tag{4}$$

B. Stage 1: Baseline LightGBM Model

The baseline model is a LightGBM gradient boosting classifier [5] trained on transaction-level features. The ensemble prediction is:

$$\hat{p}_{\text{base}}(x) = \sigma\left(\sum_{m=1}^{M} T_m(x)\right), \quad \sigma(z) = \frac{1}{1+e^{-z}} \tag{5}$$

where each tree T_m partitions the feature space into leaf regions $\{R_{m,l}\}$ with leaf output values $\{c_{m,l}\}$. The training objective minimizes binary cross-entropy with L1/L2 regularization on leaf weights:

$$\mathcal{L} = \sum_i \ell(y_i, \hat{p}_i) + \sum_m \left(\gamma L_m + \tfrac{\lambda}{2} \sum_l w_{m,l}^2\right) \tag{6}$$

LightGBM's leaf-wise tree growth strategy yields shallower, more efficient trees than level-wise growth, providing approximately 10–20× speedup on tabular data without loss of accuracy [5].

C. Stage 2: Contextual XGBoost Model

The contextual model is an XGBoost classifier [4] trained on a concatenation of transaction features and customer history aggregates. It produces:

$$\hat{p}_{\text{ctx}}(x_c) = \sigma\left(\sum_{m=1}^{M} T_m(x_c)\right) \tag{7}$$

Since ground-truth delinquency labels are often delayed (payment outcomes are known weeks to months after the transaction), PIE generates proxy labels for training the contextual model:

$$y_{\text{proxy}} = \begin{cases} 0 & \text{safe merchant and event type} \\ 0.3 & \text{high income-expense ratio, low prior score} \\ 0.7 & \text{frequent declines or elevated DPD} \\ 1.0 & \text{explicit delinquency history} \end{cases} \tag{8}$$

D. Stage 3: Fusion Engine

Both model outputs are fused using a weighted blend with disagreement dampening. Let $\Delta = |\hat{p}_{\text{base}} - \hat{p}_{\text{ctx}}|$ and $\tau = 0.25$:

$$\hat{p}_{\text{fused}} = \begin{cases} 0.7\,\hat{p}_{\text{base}} + 0.3\,\hat{p}_{\text{ctx}} & \Delta < \tau \\ \frac{\hat{p}_{\text{base}} + \hat{p}_{\text{ctx}}}{2} & \Delta \geq \tau \end{cases} \tag{9}$$

TABLE I
FEATURE SUMMARY FOR BASELINE AND CONTEXTUAL MODELS

Feature	Model	Definition
amount	Base	Transaction amount ($)
balance	Base	Post-transaction account balance
dpd	Base	Days past due on obligations
decline_count	Base	Rolling 90-day declined transactions
merchant_type	Base	Categorical merchant classification
is_international	Base	Cross-border transaction flag
spending_velocity	Base	$\sum \text{amt}_i/\text{days_active}$
decline_rate	Base	Declines / total attempts
balance_volatility	Base	$\sigma(\text{bal})/\mu(\text{bal}) + \epsilon$
merchant_diversity	Base	Shannon entropy of merchant mix
dpd_acceleration	Base	$\text{dpd}_t - \text{dpd}_{t-1}$
avg_monthly_income	Context	Mean observed income credits (12-month period)
income_expense_ratio	Context	$\mu_{\text{income}}/\mu_{\text{spend}}$
prior_delinquency_count	Context	Historical delinquency incidents
prior_risk_score_avg	Context	Mean of last N risk scores
risk_trend	Context	Linear regression slope on score history
payment_consistency	Context	Normalized deviation from mean transaction amount
balance_decrease_ratio	Context	Fraction of 20-transaction window with declining balance

Business-rule overrides are applied post-fusion to handle operationally recognized safe events:

$$\hat{p}_{\text{final}} = \begin{cases} \min(\hat{p}_{\text{fused}}, 0.3), & \text{if event} \in \{\texttt{INCOME_CREDIT}, \texttt{PAYMENT}\} \\ \min(\hat{p}_{\text{fused}}, 0.4), & \text{if income-expense ratio} > 1.5 \\ \hat{p}_{\text{fused}}, & \text{otherwise} \end{cases} \tag{10}$$

The final continuous risk score $s = 100 \cdot \hat{p}_{\text{final}}$ is mapped to four buckets, as shown in Table II.

TABLE II
RISK SCORE BUCKETING THRESHOLDS AND OPERATIONAL ACTIONS

Bucket	Score Range	Risk Level	Action
LOW	$[0, 30)$	Minimal	Log only
MEDIUM	$[30, 70)$	Moderate	Monitor
HIGH	$[70, 85)$	Elevated	Soft outreach
CRITICAL	$[85, 100]$	Severe	Immediate intervention

E. Threshold Optimization

The classification threshold θ^* is selected via F1 sweep on a held-out validation set:

$$\theta^* = \arg\max_{\theta \in [0,1]} F_1(\theta) = \arg\max_{\theta} \frac{2 \cdot \text{Prec}(\theta) \cdot \text{Rec}(\theta)}{\text{Prec}(\theta) + \text{Rec}(\theta)} \tag{11}$$

This is preferable to using a fixed 0.5 threshold in imbalanced settings typical of delinquency datasets, where default rates are typically 3–15% [3].

V. DRIFT DETECTION AND RETRAINING

A. Drift Metrics

PIE employs three complementary drift statistics evaluated over rolling 30-day windows against a baseline distribution:

Population Stability Index (PSI) [17] measures feature bin proportion shifts:

$$\text{PSI} = \sum_{i=1}^{n} \frac{(P_i^{\text{cur}} - P_i^{\text{base}})}{0.5(P_i^{\text{base}} + 0.001)} \tag{12}$$

Kolmogorov-Smirnov Statistic [16] quantifies maximal CDF divergence:

$$\text{KS} = \sup_x |F_{\text{base}}(x) - F_{\text{cur}}(x)| \tag{13}$$

Jensen-Shannon Divergence [18] provides a symmetric, bounded divergence measure:

$$\text{JS}(P\|Q) = \frac{1}{2}\text{KL}(P\|M) + \frac{1}{2}\text{KL}(Q\|M), \quad M = \frac{P+Q}{2} \tag{14}$$

Table III summarizes the three-zone classification for each metric.

TABLE III
DRIFT DETECTION THRESHOLDS AND SYSTEM RESPONSE

Metric	Green	Yellow	Red	Red Action
PSI	< 0.10	$[0.10, 0.25)$	≥ 0.25	Retrain
KS	< 0.10	$[0.10, 0.20)$	≥ 0.20	Retrain
JS	< 0.15	$[0.15, 0.30)$	≥ 0.30	Retrain
AUC Δ	> -0.02	$[-0.05, -0.02)$	< -0.05	Retrain

B. Retraining Workflow

The retraining pipeline is triggered whenever any drift metric falls into the Red zone, or manually triggered or scheduled weekly:

1) **Data Retrieval**: Load the last 60-90 days' transactions having known outcomes from the label store.
2) **Training/Test Set Preparation**: Stratified train/test set (80/20 split by outcome label).
3) **Training Model**: Train the LightGBM model with early stopping based on the validation AUC.
4) **Threshold Tuning**: Tune θ^* using the F1 score maximization.
5) **Model Saving**: Save the trained model, schema, and threshold, and append the validation metrics to the historical log.
6) **Activation**: Atomically replace the active model reference, updating the in-memory cache.
7) **Contextual Model Training**: Train a separate version of XGBoost with the extended feature matrix using proxy labels.

Each new model version is time-stamped and saved for possible rollback purposes. The training process is performed in the background APScheduler worker thread without interfering with the inference pipeline and takes approximately 5-15 minutes per node.

VI. Intervention Orchestration

PIE's intervention engine translates all the high-risk signals into operationally actionable workflows through the 4 tier workflows. The four workflow tiers (WF1–WF4), summarized in Table IV below, correspond to escalating levels of human involvement whenever required.

TABLE IV
Intervention Workflow Tiers and Triggers

WF	Score Tier	Action	Human Involved
WF1	LOW (< 30)	Log event, no outreach	No
WF2	MEDIUM (30–60)	Auto-generated payment plan via customer channel	No
WF3	HIGH (60–80)	Alert + CRM case creation	Review
WF4	CRITICAL (≥ 80)	Human agent ticket, escalated contact	Yes

Previewing, approving, rejecting, and deferring interventions is enabled for risk team members who have been authenticated using Google OAuth. Each action is time-stamped to provide an audit trail. The outcome of the intervention whether it leads to customer payment, engagement, or non-engagement is added back to the label store as ground-truth data.

VII. Experimental Evaluation

A. Datasets and Setup

The following experiments were carried out using two datasets: (1) synthetic transaction dataset created using the PIE simulator, and (2) `pie_sample_cleaned.csv` which is a cleaned corpus of transaction history inspired by real-world data, with delinquency labels. The baseline LightGBM model used a training set of 1,000,000 transactions with a default ratio of 12%. The contextual XGBoost model used a training set of 45,000 customer records. All the experiments were performed on one Ubuntu 22.04 node (8 cores, 32 GB memory).

B. Model Performance

Table V reports held-out test performance for the baseline, contextual, and fused ensemble.

TABLE V
Model Performance on Held-Out Test Set

Model	AUC	Prec.	Recall	F1	Acc.
LightGBM (Baseline)	0.871	0.763	0.801	0.782	0.847
XGBoost (Contextual)	0.952	0.912	0.943	0.931	0.947
Ensemble (Fused)	**0.912**	**0.828**	**0.872**	**0.857**	**0.897**

The fused ensemble model outperforms the stand-alone models and the logistic regression benchmark in all cases, thus illustrating the complementarity of transaction-level and customer-level information. The boost in AUC accuracy from 0.871 (LightGBM stand-alone) to 0.912 (fused ensemble) indicates a significant decrease in errors in ranking delinquency [3].

C. Latency and Throughput

Table VI reports end-to-end scoring latency and system throughput.

TABLE VI
System Latency and Throughput Benchmarks

Metric	Value
Baseline LightGBM inference	10–50 ms
Contextual XGBoost inference	50–100 ms
Feature assembly (from Redis)	5–20 ms
Database write (PostgreSQL)	10–30 ms
E2E median latency	120 ms
E2E P95 latency	380 ms
Throughput (single node)	450–520 tx/sec
Stream consumer lag (steady state)	$<$ 50 messages
Retraining duration (100K samples)	6–12 min

D. Drift Detection Sensitivity

For assessing the sensitivity of drift detection, we introduced artificial shifts to the distributions of the `amount` and `dpd` variables in week 4, 8, and 12 during the course of a 16-week simulation process. The drift detection using the PSI and KS metrics was able to detect red-zone drift for the three injection cases accurately, leading to retraining after a single monitoring period (one hour). No false positives were detected within a 30-day baseline window.

E. Intervention Impact Simulation

In our Monte Carlo simulation analysis involving 10,000 customers for 6 months, we evaluated the three scenarios: (i) control (no intervention), (ii) batch monthly intervention based on rule, and (iii) PIE intervention (real-time). Our simulation estimated intervention success probability as a function of number of days before default where contact was possible. As PIE could do a real-time scoring, contact could be made 19 days sooner on average than batch contact, resulting in 23% fewer charge-offs in comparison to batch and 41% fewer than control.

VIII. Discussion

A. Strengths

Architecture-wise, there are several clear benefits that PIE has against current solutions. In a two-model ensemble approach, PIE will gracefully fail in case the context model does not have any data to process for the new customer because the baseline can continue working in a stand-alone mode. The disagreement mitigation will ensure that contextual model outliers do not affect the fused score. Redis Streams offer

ordered events delivery in a cost-efficient manner compared to Apache Kafka [11] to be usable for mid-market players without specialized data engineers.

Drift detection via multiple metrics overcomes the shortcomings of using just one statistical feature. PI is prone to coarser changes in the distributions, KS detects fine-scale discrepancies between CDFs, and JS divergence offers an efficient, bounded measurement which is insensitive to sparse bins. These three combined decrease the false alarms' rate better than any single metric would [14].

B. Limitations and Future Work

However, there are several limitations which need to be mentioned. For the contextual model, the proxy labeling technique will create some kind of label noise, possibly leading to biased results regarding recall; hence, in future research, it is important to consider semi-supervised learning algorithms or active label acquisition strategies [22]. Current fusion weights are heuristic in nature (70/30), while using a meta-learner (stacking) algorithm [23] would allow one to find out the optimal fusion weights. Feature fairness analysis – whether the model exhibits any discriminatory behavior toward some demographic groups – is a mandatory condition in most regulation acts, but this is not implemented yet [21].

IX. Conclusion

PIE, the Pre-Delinquency Intervention Engine, exemplifies how streaming machine learning infrastructure, ensemble gradient boosting, drift monitoring, and intervention orchestration may be integrated to build a robust credit risk engine. With PIE, financial institutions will be able to take action prior to default and avoid the credit losses associated with it by moving from batch-periodic to real-time scoring. As shown from our experimental evaluations, the ensemble model we used of LightGBM and XGBoost produces an AUC of 0.912 in its ability to predict default, while delivering under-500ms latency per inference and triggering of auto-retraining when drift is detected.

References

[1] Bank for International Settlements, "Credit to the Non-Financial Sector," *BIS Statistics*, 2023. [Online]. Available: https://www.bis.org/statistics/totcredit.htm

[2] L. C. Thomas, *Credit Scoring and Its Applications*, 2nd ed. Philadelphia, PA: SIAM, 2017.

[3] S. Lessmann, B. Baesens, H.-V. Seow, and L. C. Thomas, "Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research," *European Journal of Operational Research*, vol. 247, no. 1, pp. 124–136, 2015.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, 2016, pp. 785–794.

[5] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[6] D. J. Hand and W. E. Henley, "Statistical classification methods in consumer credit scoring: A review," *Journal of the Royal Statistical Society: Series A*, vol. 160, no. 3, pp. 523–541, 2005.

[7] I. Brown and C. Mues, "An experimental comparison of classification algorithms for imbalanced credit scoring data sets," *Expert Systems with Applications*, vol. 39, no. 3, pp. 3446–3453, 2012.

[8] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*, 2015, pp. 159–166.

[9] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, "Unsupervised real-time anomaly detection for streaming data," *Neurocomputing*, vol. 262, pp. 134–147, 2017.

[10] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009, pp. 1–6.

[11] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. 6th International Workshop on Networking Meets Databases (NetDB)*, Athens, Greece, 2011.

[12] Redis Ltd., "Redis Streams," *Redis Documentation*, 2023. [Online]. Available: https://redis.io/docs/data-types/streams/

[13] R. Ganti, F. Ye, and H. Lei, "Mobile crowdsensing: Current state and future challenges," *IEEE Communications Magazine*, vol. 49, no. 11, pp. 32–39, 2011.

[14] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[15] M. G. Kelly, D. J. Hand, and N. M. Adams, "The impact of changing populations on classifier performance," in *Proc. 5th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Diego, CA, 1999, pp. 367–371.

[16] F. J. Massey, Jr., "The Kolmogorov-Smirnov test for goodness of fit," *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.

[17] B. Yurdakul, "Statistical properties of population stability index," *Western Michigan University, Dissertations*, 2018.

[18] M. L. Menéndez, J. A. Pardo, L. Pardo, and M. C. Pardo, "The Jensen-Shannon divergence," *Journal of the Franklin Institute*, vol. 334, no. 2, pp. 307–318, 1997.

[19] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[20] S. Finlay, "Credit scoring, response modelling and insurance rating: A practical guide to forecasting consumer behaviour," 2010.

[21] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv preprint arXiv:1702.08608*, 2017.

[22] B. Settles, "Active learning literature survey," *University of Wisconsin-Madison Computer Sciences Technical Report 1648*, 2009.

[23] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[24] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.